Get unlimited access to the best of Medium for less than $1/week. Become a member

Data Cleaning with SQL

Carlos da Costa · Follow
Published in Learning Data · 9 min read · Jan 3

158 1

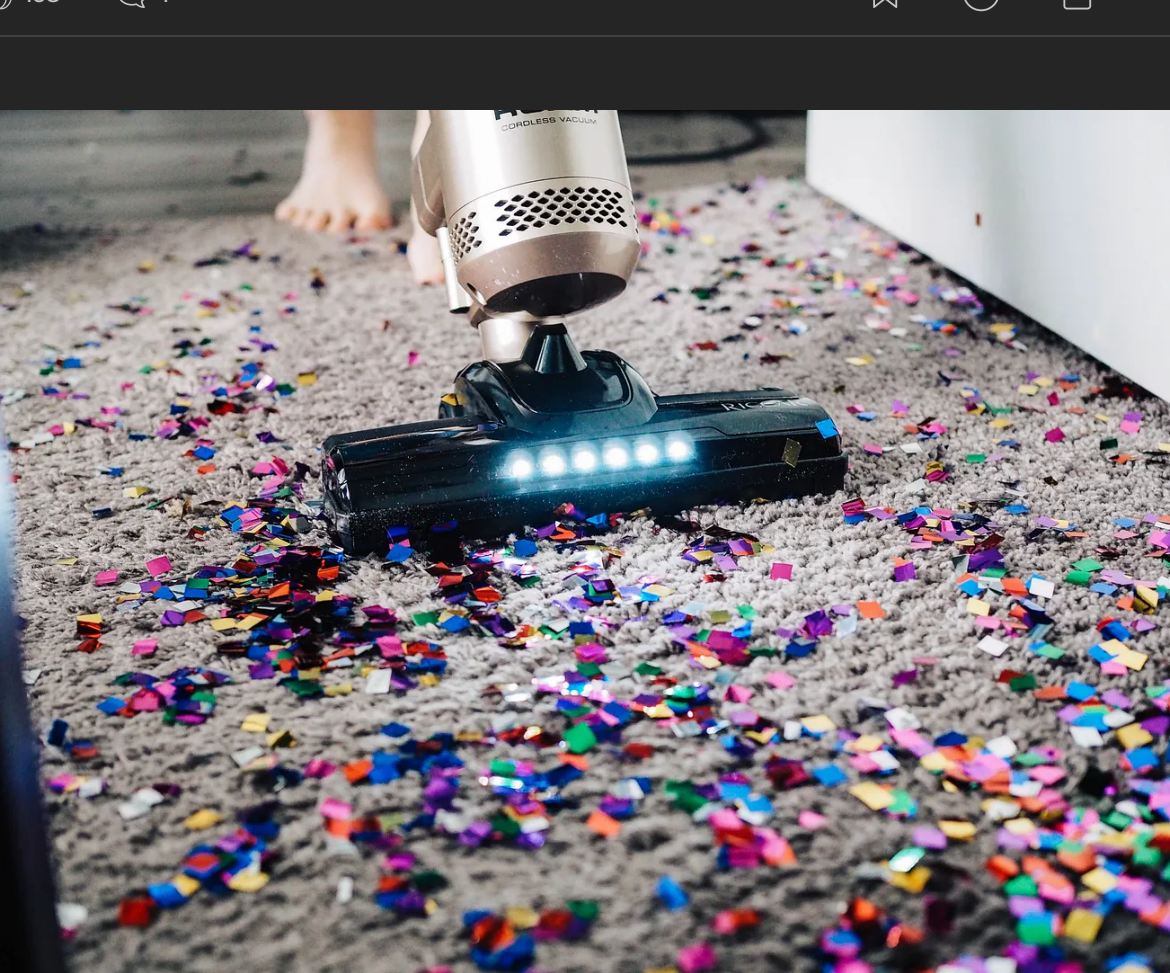


Photo by No Revisions on Unsplash

When building a machine learning model or engaging in a data analysis project, we need to ensure that the data being used is of the best quality. Remember: ***garbage in, garbage out***. Data cleaning is an essential step in any data project; it is the process of identifying and removing data inconsistencies and errors. Come along, and let's dive into data cleaning with SQL.

For this tutorial, we will be using a simple HR database with two tables, `employees` and `department` and before we dive into cleaning we should understand our data.

1. Understanding the data

Firstly, we need to understand how the database is structured. This will give us insight into the active relationships among the tables, and the data types used will consequently guide us on how to conduct our cleaning and analysis later on.

1.2 Show the tables

```
show tables;
```

```
+-----------------+
| Tables_in_hr_db |
+-----------------+
| departments     |
| employees       |
+-----------------+
```

We only have two tables in the database — lucky us! Well, not really. This is just for learning purposes. In a real-world production database, you will find much more than two tables.

1.3 Describe the tables

```
DESCRIBE departments;

DESCRIBE employees;
```

```
+-----------------+-------------+------+-----+---------+-------+
| Field           | Type        | Null | Key | Default | Extra |
+-----------------+-------------+------+-----+---------+-------+
| department_id   | int         | NO   | PRI | NULL    |       |
| department_name | varchar(50) | YES  |     | NULL    |       |
+-----------------+-------------+------+-----+---------+-------+

+---------------+--------------+------+-----+---------+-------+
| Field         | Type         | Null | Key | Default | Extra |
+---------------+--------------+------+-----+---------+-------+
| employee_id   | int          | NO   | PRI | NULL    |       |
| employee_name | varchar(100) | YES  |     | NULL    |       |
| department_id | int          | YES  | MUL | NULL    |       |
| gender        | varchar(10)  | YES  |     | NULL    |       |
| salary        | varchar(15)  | YES  |     | NULL    |       |
| hire_date     | date         | YES  |     | NULL    |       |
+---------------+--------------+------+-----+---------+-------+
```

In the script above, we described the two tables and now, we can see the data type of the columns and that there is a 1-to-many relationship between the `employees` and `department` tables.

1.4 Retrieve the data

We will display the top 5 rows to get a sense of the data stored in the tables.

```
SELECT * FROM departments LIMIT 5;
SELECT * FROM employees LIMIT 5;
```

```
+---------------+-----------------+
| department_id | department_name |
+---------------+-----------------+
|             1 | Human Resources |
|             2 | Marketing       |
|             3 | Finance         |
+---------------+-----------------+
+-------------+---------------+---------------+--------+------------+------------
| employee_id | employee_name | department_id | gender | salary     | hire_date
+-------------+---------------+---------------+--------+------------+------------
|           1 | John Doe      |             1 | M      | $60,000.00 | 2022-01-15
|           2 | Jane Smith    |             2 | Female | $75,000.00 | 2022-02-20
|           3 | Bob Johnson   |             1 | m      | $55,000.00 | 2022-03-10
|           4 | Alice Brown   |             3 | f      | $80,000.00 | 2022-04-05
|           5 | Charlie White |             2 | male   | $70,000.00 | 2022-05-12
+-------------+---------------+---------------+--------+------------+------------
```

From this, we can already observe some inconsistencies in our data. For example, the gender column lacks uniformity, as it contains different values for the same gender, and the salary column includes a dollar sign, which could pose a problem if we intend to perform calculations with it.

In the next section, we will address these issues along with others that may not have been identified yet.

2. Cleaning the data

2.1 Dealing with duplicates

Duplicates can impact negatively our analysis or model so it is a best practice to search for duplicates and deal with them early.

In the script below we will search for duplicates. let’s skip the employee_id column since we know that is a primary key from the previous step.

A simple approach using the `ORDER BY` clause.

```
SELECT employee_name, department_id, gender, salary, hire_date
FROM employees
ORDER BY 1,2,3,4,5;
```

```
+-----------------+---------------+--------+------------+------------+
| employee_name   | department_id | gender | salary     | hire_date  |
+-----------------+---------------+--------+------------+------------+
| Alice Brown     |             3 | f      | $80,000.00 | 2022-04-05 |
| Bob Johnson     |             1 | m      | $55,000.00 | 2022-03-10 |
| Bob Johnson     |             1 | M      | $55,000.00 | 2022-03-10 |
| Carlos da Costa |             3 | m      | $75,000.00 | 2050-05-10 |
| Charlie White   |             2 | male   | $70,000.00 | 2022-05-12 |
| David Black     |             1 | F      | $68,000.00 | 2022-07-25 |
| Eva Green       |             3 |        | $72,000.00 | 2022-06-18 |
| Frank Red       |          NULL | F      | $60,000.00 | 2022-09-05 |
| Jane Smith      |             2 | Female | $75,000.00 | 2022-02-20 |
| Jane Smith      |             2 | female | $75,000.00 | 2022-02-20 |
| John Doe        |             1 | M      | $60,000.00 | 2022-01-15 |
| John Doe        |             1 | m      | $60,000.00 | 2022-01-15 |
| Sophie Grey     |             2 | NULL   | $65,000.00 | 2022-08-30 |
+-----------------+---------------+--------+------------+------------+
```

This approach works only for a table with few records. Now let’s check with a more robust approach, using a subquery and aggregation COUNT function to count the number of duplicates.

```
SELECT COUNT(*)
FROM(
  SELECT employee_name, department_id, gender, salary, hire_date, count(*) as qu
  FROM employees
  GROUP BY 1,2,3,4,5) suquery
WHERE quantity > 1;
```

```
+----------+
| COUNT(*) |
+----------+
|        3 |
+----------+
```

The script above counts the number of duplicated records by selecting all the employees and groups by their similarity and counting, then filtering by only the rows that have the count greater than 1.

For more clarity, we can now list the actual duplicated records and the number of their occurrences.

```
SELECT employee_name, department_id, gender, salary, hire_date, count(*) as quan
FROM employees
GROUP BY 1,2,3,4,5
HAVING quantity > 1;
```

```
+---------------+---------------+--------+------------+------------+----------+
| employee_name | department_id | gender | salary     | hire_date  | quantity |
+---------------+---------------+--------+------------+------------+----------+
| John Doe      |             1 | M      | $60,000.00 | 2022-01-15 |        2 |
| Jane Smith    |             2 | Female | $75,000.00 | 2022-02-20 |        2 |
| Bob Johnson   |             1 | m      | $55,000.00 | 2022-03-10 |        2 |
+---------------+---------------+--------+------------+------------+----------+
```

Now that you have found the duplicates, you can decide to remove, update, or keep them based on your requirements.

2.2 Dealing with missing data and Nulls

We are going to use two scripts, one to find empty strings and another one to find the NULL values.

```
-- display record with empty strings
SELECT * FROM employees WHERE gender = "";

-- display records with null
SELECT * FROM employees
WHERE
employee_name IS NULL OR
department_id IS NULL OR
gender IS NULL OR
salary IS NULL OR
hire_date IS NULL;
```

```
+-------------+---------------+---------------+--------+------------+------------
| employee_id | employee_name | department_id | gender | salary     | hire_date
+-------------+---------------+---------------+--------+------------+------------
|           6 | Eva Green     |             3 |        | $72,000.00 | 2022-06-18
+-------------+---------------+---------------+--------+------------+------------

+-------------+---------------+---------------+--------+------------+------------
| employee_id | employee_name | department_id | gender | salary     | hire_date
+-------------+---------------+---------------+--------+------------+------------
|           8 | Sophie Grey   |             2 | NULL   | $65,000.00 | 2022-08-30
|           9 | Frank Red     |          NULL | F      | $60,000.00 | 2022-09-05
+-------------+---------------+---------------+--------+------------+------------
```

From the script above, we can see one record with an empty string in the gender column and two with NULL values.

Depending on the dataset and requirements, you can handle missing data by filling in with the average, median, mode, filling forward, onward or replacing with a constant.

Let’s say we know that Sophie Grey is a female, we could use the case statement to replace the NULL with the actual gender.

```
SELECT employee_id, employee_name, department_id, salary, hire_date,
CASE WHEN gender IS NULL then "F" END AS gender
FROM employees
WHERE employee_id = 8;
```

In case it is expected to have NULL values you can use the coalesce function to display a default value.

2.3 Dealing with wrong categorical data

From the previous sections, we have noticed that the gender column has a lot of inconsistency, with values like `Female` , `F` , `f` and so on. If we want to have quality data, we need to sort this out. Firstly, let’s print out all the distinct values in the gender column and then replace them with a proper category `m` for males and `f` for females.

We will be using the UPPER function since the DISTINCT clause is case insensitive.

```
SELECT DISTINCT UPPER(gender) AS gender FROM employees;

SELECT employee_id, employee_name, department_id, salary, hire_date, CASE
WHEN UPPER(gender) IN ("M","MALE") THEN "m"
WHEN UPPER(gender) IN ("FEMALE","F") THEN "f"
ELSE "" END AS gender
FROM employees;
```

```
+--------+
| gender |
+--------+
| M      |
| FEMALE |
| F      |
| MALE   |
|        |
| NULL   |
+--------+

+-------------+-----------------+---------------+------------+------------+-----
| employee_id | employee_name   | department_id | salary     | hire_date  | gend
+-------------+-----------------+---------------+------------+------------+-----
|           1 | John Doe        |             1 | $60,000.00 | 2022-01-15 | m
|           2 | Jane Smith      |             2 | $75,000.00 | 2022-02-20 | f
|           3 | Bob Johnson     |             1 | $55,000.00 | 2022-03-10 | m
|           4 | Alice Brown     |             3 | $80,000.00 | 2022-04-05 | f
|           5 | Charlie White   |             2 | $70,000.00 | 2022-05-12 | m
|           6 | Eva Green       |             3 | $72,000.00 | 2022-06-18 |
|           7 | David Black     |             1 | $68,000.00 | 2022-07-25 | f
|           8 | Sophie Grey     |             2 | $65,000.00 | 2022-08-30 |
|           9 | Frank Red       |          NULL | $60,000.00 | 2022-09-05 | f
|          10 | John Doe        |             1 | $60,000.00 | 2022-01-15 | m
|          11 | Jane Smith      |             2 | $75,000.00 | 2022-02-20 | f
|          12 | Bob Johnson     |             1 | $55,000.00 | 2022-03-10 | m
|          13 | Carlos da Costa |             3 | $75,000.00 | 2050-05-10 | m
+-------------+-----------------+---------------+------------+------------+-----
```

Have you noticed that there is an employee who is a time traveler?

According to the HR department, the employee Carlos da Costa was hired in 2050–05–10; use the CASE clause to modify it.

2.4 Type conversion

We have a dollar sign in the `salary` column which will prevent us from doing any calculations.

Let’s say we want to get the average salary by department, if we type in the script below we will get zeros for the average salary.

```
SELECT department_name, AVG(salary) as "Average Salary"
FROM employees LEFT JOIN departments
ON employees.department_id = departments.department_id
GROUP BY department_name;
```

```
+-----------------+----------------+
| department_name | Average Salary |
+-----------------+----------------+
| Human Resources |              0 |
| Marketing       |              0 |
| Finance         |              0 |
+-----------------+----------------+
```

To fix this, let’s first replace the dollar sign and cast the data type to decimal.

```
SELECT department_name,
AVG(
cast(replace(salary,'$','') AS DECIMAL(10,2))
)
as "Average Salary"
FROM employees LEFT JOIN departments
ON employees.department_id = departments.department_id
GROUP BY department_name;
```

```
+-----------------+----------------+
| department_name | Average Salary |
+-----------------+----------------+
| Human Resources |      59.600000 |
| Marketing       |      71.250000 |
| Finance         |      75.666667 |
+-----------------+----------------+
```

That is it, now the salary column is in the correct format and can be used for calculations.

3. Final thoughts

While there is much more to explore in the realm of data cleaning, this tutorial serves as a starting point to ignite your journey. The most effective way to enhance your skills in data cleaning and discover new tricks is through active participation in data projects. I trust you’ve got some valuable tips from this tutorial, and I encourage you to dive into real-world data scenarios to further sharpen your expertise

Good day, good coding.

...

The contents of external submissions are not necessarily reflective of the opinions or work of Maven Analytics or any of its team members.

We believe in fostering lifelong learning and our intent is to provide a platform for the data community to share their work and seek feedback from the Maven Analytics data fam.

Happy learning!

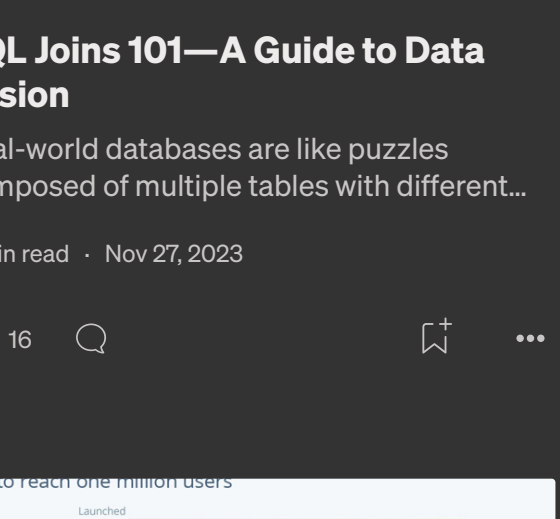
-Team Maven

Analytics Data Engineering Sql Data Data Analysis

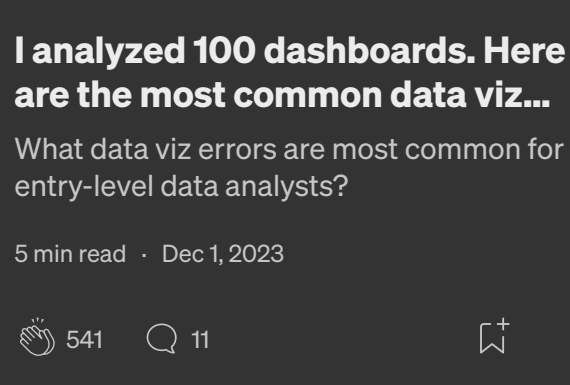
158 1

Written by Carlos da Costa
Follow
239 Followers · Writer for Learning Data
I make hard stuff about data easy to understand. Let's dive into the data world together.
https://www.linkedin.com/in/carlosdavidcosta

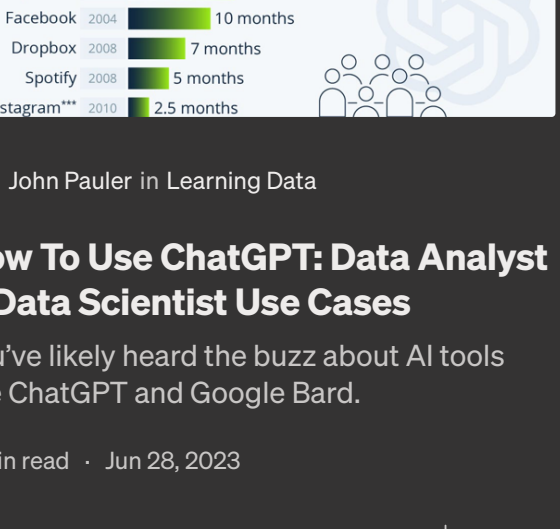
More from Carlos da Costa and Learning Data



Carlos da Costa
SQL Joins 101—A Guide to Data Fusion
Real-world databases are like puzzles composed of multiple tables with different...
6 min read · Nov 27, 2023
16



MargaretEfron in Learning Data
I analyzed 100 dashboards. Here are the most common data viz...
What data viz errors are most common for entry-level data analysts?
5 min read · Dec 1, 2023
541 11



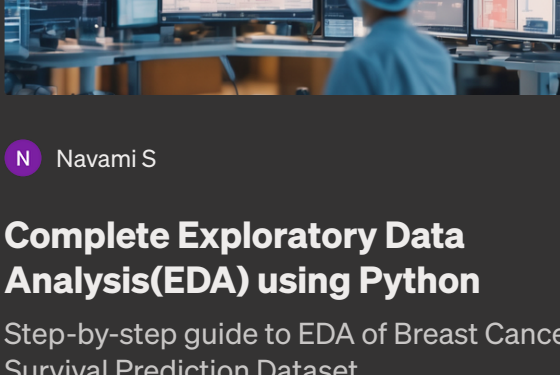
John Pauler in Learning Data
How To Use ChatGPT: Data Analyst & Data Scientist Use Cases
You've likely heard the buzz about AI tools like ChatGPT and Google Bard.
8 min read · Jun 28, 2023
662 12

Carlos da Costa
How to Get Started with SQL Querying
Are you embarking on a journey to become a data analyst, data scientist, or software...
7 min read · Nov 10, 2023
23

See all from Carlos da Costa See all from Learning Data

Recommended from Medium

Anjolaoluwa Ajayi in AI monks.io
130 Data Science Terms Every Data Scientist Should Know
Most Data Science Jargon explained in plain English
11 min read · Jan 4
1K 10



Navami S
Complete Exploratory Data Analysis(EDA) using Python
Step-by-step guide to EDA of Breast Cancer Survival Prediction Dataset
19 min read · Jan 7
512 5

Lists

Apple's Vision Pro
7 stories · 45 saves

Practical Guides to Machine Learning
10 stories · 938 saves

ChatGPT
23 stories · 403 saves

Predictive Modeling w/ Python
20 stories · 807 saves

Regan Muthomi
The Ultimate SQL Cheatsheet
A few months ago, I hosted SQL learning sessions with a tech community in Nairobi,...
8 min read · Oct 16, 2023
122

Prayag Padwal
ADIDAS Sales Analysis
Unraveling the Secrets of Adidas' Sales Success: A Data-Driven Analysis
14 min read · Dec 28, 2023
364 1

Williams Nse Idara
Adidas Sales Analysis: Unveiling Insights through Data-driven...
Analyzing the Adidas US Sales dataset from different aspects so as to derive meaningful...
20 min read · Oct 31, 2023
426 8

SQL Fundamentals in DevOps.dev
Data Cleaning: 10 Essential SQL Queries for Data Scientists
Effective data cleaning is a crucial step in the data science pipeline. SQL, a powerful...
✦ · 2 min read · Jan 7
77 1

See more recommendations